JavaScript Lecture Notes - Day 6 (Morning)

Regular Expressions • Interacting with a Server (fetch) • Showing Server Data in a Table with the DOM • JSON Object Tweaks

Lecture date: 26 October 2026 | Instructor: Abhishek Purohit

> **Extra (not said in the lecture):** This session continues the form-validation work from the previous lecture (where the instructor promised 'format' validation of the password). Boxes labelled **Instructor's point** are from the lecture. Boxes labelled **Extra** are my additions and were **not** said in the lecture. Your own notes and code files are woven into the sections; where your final code differs slightly from what was typed live, I follow your version and say so.

Contents

Part	Topics
A	Regular expressions (regex): what, why, where to get them, using them in JS with `test()`, password validation on submit and on key-up, phone / email regex
B	Interacting with a server: why asynchronous, no page reload, the `fetch` function, URL + method, JSONPlaceholder, response object, `response.json()`, then/catch vs async/await, error responses
C	Displaying server data in an HTML table using the DOM (loop, createElement, appendChild, destructuring)
D	JSON object tweaks: variable as a value, variable name as a key, bracket access, variable value as a key
E	Revision: summary, definitions, syntax, differences, mistakes, interview Q&A, exercises, checklist, source-coverage check

PART A - Regular Expressions (Regex)

1. Why regular expressions?

In the previous session we validated 'required' fields. The next requirement is to check whether a string follows a **given format / pattern**, for example: 8 to 12 characters, at least one uppercase letter, at least one digit, no symbols allowed. Writing this with plain string-manipulation logic becomes complex.

> **Instructor's point:** Instead of writing complex logic for string manipulation, use a **regular expression**. It is a formula-like expression that contains the **rules** for your format. You do not write any logic - you create a regular expression and just **test your string against it**.

1.1 Definition

- A regular expression (regex) is an expression / formula that contains some rules - for example the maximum length of a string, the minimum length, 'at least one uppercase letter', etc.
- We use regex to **compare strings based on a pattern / format**.
- If the string satisfies **all** the rules, the comparison gives **true**; if any rule is violated (invalid data), it gives **false**.
- Regex is used in other technologies too, and it stays the same - it is not specific to JavaScript.
- It is especially useful for complex validations in forms.

1.2 Creating regexes - not the topic today

How to *write* a regex is a totally different, in-depth concept and takes time to learn. In this lecture we only learn **how to use** a regex and compare a string with it.

- Many tools / libraries give regular expressions out of the box. The website **regexlib** was shown; you can also search 'regex for password'.
- You can describe your requirement to a GenAI (the instructor named GPT-4 and Gemini) and it will generate the regex.
- Regexes can be customised: e.g. 'between 8 and 20 characters, one uppercase, one lowercase, one number, one special character, and spaces explicitly forbidden'.
- As a developer you can take reference from predefined websites and AI, then simply use the regex.

2. The password regex used in the lecture

The expression chosen from the search results validates: **at least 8 characters, at least one uppercase, one lowercase, one number and one special character.** This is the same regex as in your code file:

```
const passwordRegex =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;
```

2.1 Rules for writing a regex in JavaScript

- Enclose the entire expression inside **slashes**: one `/` to open and one `/` to close.
- **Do not put it in double quotes.** Inside quotes it is just a string, and will not be treated as a regex.
- The expression normally **starts with `^` (caret)** and **ends with `$` (dollar symbol)**.

> **Extra (not said in the lecture):** In the lecture the instructor only pointed out the slashes, the caret at the start and the dollar at the end. He did not explain the inside of the regex. The table below is my explanation to help you understand your own code.

Part of the regex	Meaning (extra explanation)
`^` ... `$`	Start and end of the string - the pattern must match the whole string.
`(?=.*[a-z])`	There must be at least one lowercase letter somewhere.
`(?=.*[A-Z])`	There must be at least one uppercase letter somewhere.
`(?=.*\d)`	There must be at least one digit.
`(?=.*[@$!%*?&])`	There must be at least one special character from this set.
`[A-Za-z\d@$!%*?&]{8,}`	Only these characters are allowed, and the length must be 8 or more.

3. Using a regex: the test() function

Call `test()` on the regex object and pass the string you want to compare. It returns a Boolean.

```
const passwordRegex =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

console.log(passwordRegex.test("abc"));        // false - rules are not satisfied
console.log(passwordRegex.test("ASD@12345"));  // false - no lowercase letter
console.log(passwordRegex.test("Asd@12345"));  // true  - satisfies every rule
```

Expression	Result	Why
`regex.test(str)`	`true`	The string fulfils the entire regex.
`regex.test(str)`	`false`	The string fails at least one rule.

Expression	Result	Why
`!regex.test(str)`	`true` when invalid	The `!` (not) is used to run the error logic.

> **Extra (not said in the lecture):** The outputs of the three `console.log` lines above are my own illustration (the lecture used the form instead of the console).

4. Password validation in the form (on submit)

The sign-up form from the last lecture is extended. Two steps of validation are used for the password:

1. If the password length is 0, show **'Password is required'**.
2. **Else if** a value is present but does not satisfy the regex, show a format error message.

Why `else if`? If the length is zero there is no point comparing against the regex - show 'required' directly. Only when there is some value should the regex comparison be done. (It depends on the kind of validation you need; the instructor chose these two steps.)

```
function validate() {
    let isValid = true;

    let username = document.getElementById("username-field").value;
    let password = document.getElementById("password-field").value;
    let phone = document.getElementById("phone-field").value;
    const passwordRegex =
        /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

    if (username.length === 0) {
        document.getElementById("username-error").innerHTML = "Username is required";
        isValid = false;
    }

    if (password.length === 0) {
        document.getElementById("password-error").innerHTML = "Password is required";
        isValid = false;
    }
    else if (!passwordRegex.test(password)) {
        document.getElementById("password-error").innerHTML =
            "Required at least one uppercase, one lowercase, one number, "
            + "one special character, 8 characters";
        isValid = false;
    }

    if (phone.length === 0) {
        document.getElementById("phone-error").innerHTML = "Phone is required";
        isValid = false;
    }

    return isValid;
}
```

4.1 Execution flow

1. User clicks Register; the form's `onsubmit="return validate()"` calls `validate()`.
2. `isValid` starts as `true`. Each failed check sets it to `false` (one `return` at the end so every check runs - the earlier technique).
3. For the password, `else if (!passwordRegex.test(password))` is true when the format is violated; then the error message is shown and `isValid = false`.
4. If nothing failed, `isValid` is still `true`, so `action` runs.

4.2 Output seen in the lecture

Password typed	Result on clicking Register
(empty)	'Password is required' - first validation works.
a short / weak value	The format message 'Required at least 8 characters, uppercase, lowercase, number, special character'.
`ASD@12345` style value (capital A, S, D then @12345 - the lecture typed it exactly like this)	**No password error**; only username and phone errors (if empty). The password now fulfils the regex as the instructor observed.

> **Extra (not said in the lecture):** My caution: the lecture said this value passes. With the regex in your code it contains no lowercase letter, so I would expect it to fail. Test it yourself; `Asd@12345` definitely passes.

5. Password validation while typing (keyup event)

Validation should happen not only at submit but also as the user types (the `onkeyup` event of the password field, as in your form code). Copy the same regex and the same `else if` into `validatePassword()`.

```
function validatePassword() {
    let password = document.getElementById("password-field").value;
    const passwordRegex =
        /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

    if (password.length === 0) {
        document.getElementById("password-error").innerHTML = "Password is required";
    }
    else if (!passwordRegex.test(password)) {
        document.getElementById("password-error").innerHTML =
            "Required at least one uppercase, one lowercase, one number, "
            + "one special character, 8 characters";
    }
    else {
        document.getElementById("password-error").innerHTML = "";   // valid -> clear
    }
}
```

- **No `isValid` variable is needed** inside the keyup function (the instructor pointed this out) - it only shows or clears messages.
- Empty field: 'Password is required'. As the user starts typing: the format error appears and stays.
- As soon as the typed value fulfils the regex, the error message is **removed**.

> **Extra (not said in the lecture):** The final `else` that clears the message was the behaviour shown from the previous lecture ('message should disappear when the value becomes valid'). It is written out here to make the flow complete.

6. Phone number and email regexes

- You can search 'regex for the Indian phone number'. The result shown: an expression that begins with **+91**, has **10 digits in total**, and the number starts with **6, 7, 8 or 9**.
- Similarly there is a regex for **email** and for other fields. The same concept validates your entire form.
- Students were asked to implement the **phone validation** themselves.

```
// EXTRA (not shown in the lecture): one common way to express the Indian mobile rule
const phoneRegex = /^(\+91)?[6-9]\d{9}$/;

phoneRegex.test("9876543210");     // true
phoneRegex.test("+919876543210");  // true
phoneRegex.test("1234567890");     // false (does not start with 6-9)
```

> **Instructor's point:** Using ready-made regexes saves a lot of time compared with writing a long manual validation. Later, in React, form validation will be simplified further with a library.

PART B - Interacting with a Server (fetch)

7. Why talk to a server, and why asynchronously?

So far we handled forms and updated the page with DOM manipulation. Now we learn how a front end **sends a request to a server and gets the response**. A front end cannot do anything by itself: to fetch, save, update or delete data it has to interact with the server.

> **Instructor's point:** **Interact with a server** = send a request to a server and get the response.

Why must this process be **asynchronous**?

1. It should **not block other processes** running in the browser (a student said it should not make another request wait; the instructor extended it to all other processes).
2. It should happen **behind the scenes, without reloading the browser page**.

- **Example - Flipkart / Amazon search:** as you type 'Oppo' or 'iPhone', results appear immediately in the search window. The data comes from the **backend**, response arrives as each character is typed, and the browser does not reload.
- **YouTube:** searching or clicking a video does not reload the browser; content simply appears.
- **Older applications** reloaded the browser on every button click to bring data from the server. Modern applications must not.

8. The fetch() function

JavaScript gives a predefined function, `fetch`, to send a request and get the response **asynchronously**. It needs two things: the **URL** of the server and the **request method** (GET, POST, PUT, DELETE).

```
fetch(url);                              // GET request - default

fetch(url, { method: "POST" });          // second argument: configuration object
fetch(url, { method: "PUT" });
fetch(url, { method: "DELETE" });
```

- If no method is mentioned, `fetch` **by default uses the GET method**.
- To use POST, PUT or DELETE pass a second argument - a **configuration object** (how you are configuring the request) with the key `method`.
- To send a plain GET request, just put the URL.

9. JSONPlaceholder - a fake / dummy server

We do not have our own server, so a dummy server is used: **JSONPlaceholder** (the instructor mentioned there are many such websites, where you can even define your own data structure). It keeps static data for posts, comments, albums, photos, to-dos and **users**.

- It gives a URL for each resource, e.g. the users URL: `https://jsonplaceholder.typicode.com/users`.
- Sending a request to the users URL returns an **array of JSON objects** (one object per user).
- Real servers work the same way: backend developers provide URLs to save, fetch, update and delete data, and you as the front-end developer send requests to those URLs.

> **Instructor's point:** Each user object has: ID, name, username, email, address (a **nested object**) and phone number (and more). The table later uses id, name, username, email and phone.

10. What does fetch() return?

Not the data - `fetch` returns a **Promise**. The promise must be handled (as learned yesterday) with `then` / `catch` or with `async` / `await`. The choice is yours.

10.1 Version 1 - then / catch (your first code file)

```
<html>
    <head></head>
    <body>

        <script>
            const responsePromise = fetch("https://jsonplaceholder.typicode.com/users");
            responsePromise
            .then((response) => {
                console.log(response);
                console.log(response.status);
                const data = response.json();   // gives the data which is in the response
                data.then((users) => {
                    console.log(users);
                });
            })
            .catch((error) => {
                console.log(error);
            });
        </script>
    </body>
</html>
```

- `responsePromise` is not the response - it is the **promise** of the response.
- If the request succeeds (response arrives), the promise is **resolved** and `then` runs.
- If something fails (server error, network rejection), the promise is **rejected** and `catch` runs. Here we only log it; in real React/Angular apps you would show a message like 'something went wrong'.

10.2 The response is a complete object, not only the data

Recalling the first session: a server does not send only data. The `response` object contains lots of information about the server's reply:

Part of response	Meaning
`body`	The data (the array of JSON objects).
`headers`	Response headers.
`status`	Status code, e.g. 200 (OK), 404 (Not Found).
`statusText`	Status message text.
`type`, `url`	Type of the response and the URL it came from. (Size, content type etc. can also be found.)

Reading it: the status is a direct property - `response.status`. The data needs the function `response.json()`.

10.3 response.json() returns a promise too

- `response.json()` gives the data in the response - but it **returns another promise**.
- So `let data = response.json();` makes `data` a promise object, not the array of users.
- Handling it with `data.then((users) => {...})` inside the first `then` gives **nesting of callbacks again**. Better: return the promise and chain another `then`, or use **async / await**.

```
// Chaining instead of nesting (extra clean-up of Version 1)
fetch("https://jsonplaceholder.typicode.com/users")
    .then((response) => {
        console.log(response.status);
        return response.json();            // return the promise
    })
    .then((users) => console.log(users))
    .catch((error) => console.log(error));
```

> **Extra (not said in the lecture):** The chained version above is my tidy-up following yesterday's chaining lesson; the lecture showed the nested form first and then moved to async/await.

11. Version 2: async / await with try-catch

Create a function (e.g. `fetchUsersData`), put the code in a `try`, and use `await`:

```
async function fetchUsersData() {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
        console.log(response);                 // the whole response object
        console.log(response.status);          // e.g. 200

        const usersData = await response.json();   // await again for the data
        console.log(usersData);                    // array of user objects
    } catch (error) {
        console.log(error);
    }
}

fetchUsersData();
```

- `await fetch(...)` makes the rest of the statements wait, and when the promise resolves it returns the **response object**.
- You must wait for the response before reading its data, so `await response.json()` returns the **users array**.
- Any rejected promise sends control to `catch`.
- The function needs the `async` keyword because it uses `await`.
- The code is much easier to read, work with and scale than nested `then`s.

11.1 What about error responses (404, 500)?

A student asked: if the server returns 404, will the `catch` block run? The instructor's answer: the response **does arrive**, but it will be considered an **error response**. Only responses in the **2xx series** are success responses; 4xx and 5xx are always treated as failure responses.

> **Extra (not said in the lecture):** Extra detail: `fetch` only rejects its promise on network-level failures. For a 404 or 500 the promise **resolves**, so `catch` does not run by itself. That is why your final code checks `response.ok` and throws an error - see the next section.

PART C - Showing Server Data in a Table (DOM Manipulation)

12. Goal and plan

Use DOM manipulation (from yesterday) to display the fetched users in an HTML table. A table has `thead` (column headings) and `tbody` (data rows). The `tbody` starts **empty** and gets filled dynamically when the user clicks a button.

1. Create the table, headings and a button that calls `fetchUsersData()`.
2. Fetch the users array with `await`.
3. Loop through the array. For each user create a `tr`.
4. Create the `td` cells (5 per row), set their content with `innerHTML`.
5. Append the `td`s into the `tr`, and the `tr` into the `tbody`.

12.1 The HTML

```
<button onclick="fetchUsersData()">Fetch Users Data</button>

<table border="1" cellspacing="0" cellpadding="10">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>Phone</th>
        </tr>
    </thead>

    <tbody id="tbody">
    </tbody>
</table>
```

- `border="1"`, `cellspacing="0"` and `cellpadding="10"` (space inside a cell) were set as attributes.
- The function is no longer called separately - it is called from the button's `onclick`.
- The `tbody` has `id="tbody"` so we can get it from JavaScript.
- Number and names of columns (ID, name, username, email, phone) are chosen from the JSON object's keys. Other data, like address, can be shown the same way.

Clicking the button already fetches data into the console. The browser does **not reload** - this is the asynchronous, behind-the-scenes loading discussed earlier.

13. The full code (your final version)

```
<script>
    async function fetchUsersData() {
        try {
            const response = await fetch("https://jsonplaceholder.typicode.com/users");

            console.log(response);
            console.log(response.status);

            if (!response.ok) {
                throw new Error("HTTP Error: " + response.status);
            }

            const usersData = await response.json();
            console.log(usersData);

            let tbody = document.getElementById("tbody");

            // Clear the table before adding data again
            tbody.innerHTML = "";

            for (let i = 0; i < usersData.length; i++) {
                let row = document.createElement("tr");

                let idColumn = document.createElement("td");
                let nameColumn = document.createElement("td");
                let usernameColumn = document.createElement("td");
                let emailColumn = document.createElement("td");
                let phoneColumn = document.createElement("td");

                const { id, name, username, email, phone } = usersData[i];

                idColumn.innerHTML = id;
                nameColumn.innerHTML = name;
                usernameColumn.innerHTML = username;
                emailColumn.innerHTML = email;
                phoneColumn.innerHTML = phone;

                row.appendChild(idColumn);
                row.appendChild(nameColumn);
                row.appendChild(usernameColumn);
                row.appendChild(emailColumn);
                row.appendChild(phoneColumn);
                tbody.appendChild(row);
            }

        } catch (error) {
            console.log(error);
        }
    }
</script>
```

> **Extra (not said in the lecture):** Two things in your final version go slightly beyond what was typed live: the `response.ok` check that throws an error for non-2xx responses, and `tbody.innerHTML = ""` to clear old rows so that clicking the button twice does not duplicate the rows.

14. Step-by-step explanation

14.1 How many rows? A loop over the array

`usersData` is an **array**. For 10 users we need 10 `tr`s; for 100 users, 100 `tr`s. So the number of rows depends on the data - a loop from index 0 up to the last element creates one `tr` per iteration. (A `forEach` could also be used; the instructor used the classic `for` loop.)

> **Extra (not said in the lecture):** In the lecture the loop condition was spoken as 'less than equal to `usersData.length`'. Your code correctly uses `i < usersData.length`. With `<=` the last iteration would read `usersData[usersData.length]`, which is `undefined`, and destructuring it throws an error.

14.2 Creating the elements

- `document.createElement("tr")` creates a row (we saw `createElement` yesterday). It exists only in memory; it does **not** appear automatically in the `tbody`.
- Five `td` elements are created per row: id, name, username, email, phone. All are `td`; only their reference names differ.
- The `tbody` is obtained with `document.getElementById("tbody")`.

14.3 Putting the data in and attaching everything

Step	What happens	Where it goes
1	`createElement("tr")`	Row (in memory)
2	`createElement("td")` x 5	Columns (in memory)
3	`innerHTML = value` on each td	Cell shows the user's data
4	`row.appendChild(td)` x 5, in order	Columns become children of the **row**
5	`tbody.appendChild(row)`	Row becomes child of the **tbody**

> **Common mistake:** Order of appending: columns -> into the row; row -> into the tbody. (The instructor briefly typed `row.appendChild` for the last step and corrected himself to `tbody.appendChild(row)` - the tbody is the parent.)

14.4 Object destructuring to shorten the code

Writing `usersData[i].id`, `usersData[i].name`, ... for every field is long. Instead **destructure** the object: take out the properties you need directly into variables with the same names.

```
// long way
idColumn.innerHTML = usersData[i].id;

// destructuring - take the keys out of the current object
const { id, name, username, email, phone } = usersData[i];
idColumn.innerHTML = id;
```

The variable names must match the key names of the JSON object. After destructuring, you refer directly to `id`, `name`, etc.

14.5 Result

Click 'Fetch Users Data': the table fills with all users, and the page is never reloaded. Reload the page and the `tbody` is empty again until the button is clicked.

15. Questions answered in class

Question	Instructor's answer
Why such a lot of effort?	This 'pain' is shown on purpose: libraries/frameworks like React and Angular make this far easier (and form validation too, with a library). You appreciate it after feeling the difficulty - 'no pain, no gain'.
What if we do not know the number of columns?	You decide manually. Otherwise, if every object in the array has the **same keys**, you can read the keys from an object and loop to create that many columns.
What if the objects have different keys?	Then this logic fails (one object with 3 keys, another with 5, or `id` vs `rollNumber`), because you cannot decide the column order.

PART D - JSON Object Tweaks

A set of small tricks about JSON objects (JavaScript objects) that you may need while scaling. (Created in `json_demo.html`.)

16. A variable can be used as a value

A key's value does not have to be hard-coded; it can be a variable (for example, user input from a form, or data arriving from a file or another source). You can use variables for **one or all** keys.

```
let productPrice = 3456;

let product = { id: 1011, name: "Headphone", qty: 4, price: productPrice };
// variable can be referred as a value

console.log(product);
```

Output

```
{id: 1011, name: 'Headphone', qty: 4, price: 3456}
```

- The value of `price` is taken from the variable `productPrice` (3456).
- Typical use: four input fields (id, name, quantity, price) - all four values would come from variables.

17. Variable name as the key (shorthand)

If you write only the variable name inside the object, with no `key: value`, then **the variable's name becomes the key** and the variable's value becomes the value.

```
let productPrice = 3456;
let pid = 6541;
let pname = "Headphone";
let qty = 5;

let product = { pid, pname, productPrice, qty };   // variable name as a key

console.log(product);
```

Output

```
{pid: 6541, pname: 'Headphone', productPrice: 3456, qty: 5}
```

- `{ productPrice }` means the key `productPrice` with value 3456 (note the key is **not** `price`).
- Useful when the variable names are exactly the key names you want - you avoid writing `pid: pid, pname: pname, ...`.
- You may meet this syntax in code: it is not a collection of variables; it just uses the variable names as keys.
- Not always necessary: when you want different key names, use the normal `key: value` form.

18. Reading values: dot notation and square brackets

Besides `product.pid`, you can refer to keys like array indexes, but with the **key name** as a string inside the square brackets.

```
console.log(product["pid"],   product.pid);      // 6541 6541
console.log(product["pname"], product.pname);    // Headphone Headphone
```

Style	Example	Note
Dot notation	`product.pid`	Simple and common.

Style	Example	Note
Square-bracket notation	`product["pid"]`	Same result - another way of reading keys. The choice is yours.

19. Variable value as the key (computed key)

This time the **value** is hard-coded (or dynamic), but the **key** should come from a variable's value. Put the variable in **square brackets**.

```
let xyz = "price";
let obj = { [xyz]: 1000 };      // variable value as a key

console.log(obj);
```

Output

```
{price: 1000}
```

- `xyz` holds the string `"price"`. `[xyz]` makes the key `price`.
- If you wrote `{ xyz: 1000 }` the key would literally be `xyz`.
- Both the key (from `[xyz]`) and the value can be dynamic, or the value can be hard-coded.
- Use it when keys have to be created dynamically.

Goal	Syntax	Resulting key
Variable value as a value	`{ price: productPrice }`	`price`
Variable name as the key	`{ productPrice }`	`productPrice`
Variable value as the key	`{ [xyz]: 1000 }`	the value stored in `xyz`

20. Wrap-up and tasks given

- Practise using regular expressions: **validate a form with regex** (password, phone, email, etc.).
- Create a **sign-up form** on the same theme as the CSS form you implemented earlier, and apply the form validations with error messages shown in a better way.
- Try: if the value is **invalid, mark the border red**; if **valid, mark it green**.
- Implement the **phone validation** yourself (regex for the Indian number).
- Evening session at 7 pm; labs at 9:30.

PART E - Revision Material

21. Concise summary

1. A regex is a formula containing the rules of a string format. Use it to compare strings against a pattern.
2. In JS, write a regex between slashes (not quotes), usually starting with `^` and ending with `$`. `regex.test(str)` returns true/false.
3. Use `if (length === 0)` for 'required', and `else if (!regex.test(value))` for the format error. Validate on submit (with `isValid`) and on keyup (without `isValid`).
4. Front ends interact with servers by sending a request and reading the response - asynchronously, without blocking and without reloading the page.
5. `fetch(url)` sends a GET by default; `fetch(url, { method: "POST" })` for other methods.
6. `fetch` returns a promise. The resolved value is a response object (status, headers, body, ...). `response.json()` returns another promise that gives the data.

7. Prefer `async`/`await` with try/catch over nested `then`.
8. Success responses are 2xx; 4xx and 5xx are error responses.
9. Fill a table dynamically: loop over the array; per item create `tr` and `td`s, set `innerHTML`, append `td`s to `tr` and `tr` to `tbody`.
10. Destructuring `const { id, name } = obj` takes properties out of an object by name.
11. JSON tweaks: variable as value; `{ pid }` variable name as key; `obj["key"]` access; `{ [xyz]: 1 }` variable value as key.

22. Important definitions

Term	Definition	Purpose / key idea
Regular expression (regex)	Expression containing rules for a string format.	Validate patterns without writing complex logic.
`test()`	Method on a regex that returns true if the string fulfils it.	Compare a string against a regex.
Interact with a server	Send a request to a server and get the response.	Fetch / save / update / delete data.
`fetch()`	Predefined function to send an async request; returns a promise.	Server communication without page reload.
Configuration object	Second argument to `fetch` (e.g. `{ method: "POST" }`).	Choose the request method.
Response object	Complete server reply: body, headers, status, statusText, type, url.	Not only data.
`response.json()`	Reads the response data; returns a promise.	Get the array / object data.
JSONPlaceholder	A fake server giving dummy JSON data.	Practice fetching data.
Destructuring	Taking properties out of an object into variables of the same name.	Shorter code.

23. Syntax cheat sheet

Task	Syntax
Create a regex	`const re = /^...$/;` (slashes, no quotes)
Test a string	`re.test(value)` (true / false)
Format error check	`else if (!re.test(value)) { ... }`
GET request	`fetch(url)`
Other methods	`fetch(url, { method: "POST" })`
Status code	`response.status`
Read data	`const data = await response.json();`
Treat non-2xx as error	`if (!response.ok) throw new Error("HTTP Error: " + response.status);`
Create / append	`document.createElement("tr")`, `parent.appendChild(child)`
Destructure	`const { id, name } = usersData[i];`
Variable name as key	`{ pid, pname }`
Bracket access	`obj["key"]`
Variable value as key	`{ [xyz]: 1000 }`

24. Key differences between related concepts

Concept A	Concept B	Difference
String logic validation	Regex validation	Manual complex logic vs one rule-formula tested with `test()`.
`"..."` string	`/.../` regex	Quotes make it a string; slashes make it a regular expression.
Validate on submit	Validate on keyup	Submit needs `isValid` and `return`; keyup only shows / clears the message.
`fetch(url)`	`fetch(url, {method})`	Default GET vs explicit method.
`response`	`response.json()`	Whole reply object vs the data (a promise).
`then` chain	`async/await`	Chained callbacks vs readable sequential code.
`createElement`	`appendChild`	Create in memory vs insert into the DOM.
`product.pid`	`product["pid"]`	Same result, two ways of reading a key.
`{ price: p }`	`{ p }`	Key `price` vs key named after the variable.
`{ xyz: 1 }`	`{ [xyz]: 1 }`	Key is the text `xyz` vs key is the value stored in `xyz`.

25. Common mistakes to avoid

Mistake	Why / correction
Putting the regex in double quotes	It becomes a string; use slashes `/.../`.
Forgetting `^` and `$`	The pattern could match only a part of the string.
Running the regex when the field is empty	Check length first (`else if`) so the right message shows.
Using `isValid` and `return` inside the keyup validator	Not needed - only show / clear the message.
Treating `fetch()` as returning the data	It returns a promise. Use `await` or `then`.
Treating `response.json()` as the data	It also returns a promise - `await` it.
Expecting `catch` to run for a 404	The promise resolves. Check `response.ok` and throw.
Nested `then`s for `response.json()`	Return the promise and chain, or use async/await.
Not appending created elements	They stay in memory. `tbody.appendChild(row)`.
Appending the row to the row	Columns go into the row; the row goes into the tbody.
Using `i <= array.length`	One extra iteration with `undefined`. Use `i < array.length`.
Different keys in array objects	The simple column logic needs the same keys in every object.
`{ xyz: 1 }` instead of `{ [xyz]: 1 }`	Without brackets the key is literally `xyz`.

26. Interview questions and answers

> **Extra (not said in the lecture):** Revision questions based on the lecture's concepts; the answers' wording is mine.

Question	Answer
What is a regular expression?	An expression holding rules for a string format (length, uppercase, digits, ...) used to validate strings.
How do you test a string against a regex in JS?	`regex.test(string)` - returns true or false.
Why use a regex for validation?	It replaces complex string-manipulation logic with one pattern.
Why use `else if` for the regex check?	If the field is empty, show 'required' and skip the regex check.

Question	Answer
How does a front end talk to a server without reloading?	Asynchronously, with `fetch` (or similar), then update the page with DOM manipulation.
Why must server calls be asynchronous?	So they do not block other browser processes and the page does not reload.
What does `fetch` return?	A Promise that resolves to a response object.
Default method of `fetch`?	GET.
How do you send POST / PUT / DELETE with `fetch`?	Pass a config object: `{ method: "POST" }`.
What is in the response object?	Body (data), headers, status, statusText, type, url, ...
How do you read the response data?	`await response.json()` - json() returns a promise.
Which status codes are success?	The 2xx series; 4xx and 5xx are error responses.
How do you build table rows dynamically?	Loop over the array; create `tr`/`td`, set `innerHTML`, append `td`s to `tr`, `tr` to `tbody`.
What is object destructuring?	`const { id, name } = obj;` extracts properties into variables.
What does `{ pid, pname }` mean?	Shorthand: keys `pid` and `pname` with values of the variables.
How do you use a variable's value as an object key?	Square brackets: `{ [xyz]: 1000 }`.

27. Practice exercises (solutions not included)

Easy

1. Write a regex that accepts only exactly 6 digits and test three strings with `test()`.
2. Use `fetch` with the JSONPlaceholder users URL and log `response.status`.
3. Create an object using variables `a`, `b` and `c` with the shorthand syntax.

Medium

1. Add phone validation (Indian number) to the sign-up form, on submit and on keyup.
2. Add an email regex validation with error messages.
3. Fetch users with async/await and display only name and email in a table.
4. Make the `fetch` function throw an error for non-2xx and show the error text on the page.

Challenging

1. Build the sign-up form on the CSS theme, with validation, error messages and a **red border for invalid / green border for valid** input.
2. Fetch JSONPlaceholder **posts** and show id, title in a table; clear the table before each fetch.
3. Read the keys of the first object in the array and create the table headings dynamically (assuming all objects have identical keys).
4. Create an object with a dynamic key from a variable and read it back with square brackets.

28. Final revision checklist

- [ ] I can explain what a regular expression is and why it is used.
- [ ] I can write a regex between slashes and test a string with `test()`.
- [ ] I can validate a password with 'required' + 'format' messages, on submit and on keyup.
- [ ] I understand why server communication must be asynchronous and reload-free.
- [ ] I can use `fetch(url)` and `fetch(url, { method })`.

- [ ] I know that fetch returns a promise, and `response.json()` also returns a promise.
- [ ] I can use async/await with try/catch for fetch and check `response.ok`.
- [ ] I know the response object contains more than data (status, headers, body, ...).
- [ ] I can fill a table from an array using createElement, innerHTML and appendChild.
- [ ] I can use destructuring to shorten property access.
- [ ] I can use a variable as a value, variable name as key, bracket access, and variable value as key.

29. Source-coverage check

Completeness check against the transcript, in lecture order:

Topic discussed in the lecture	Covered in section
Why regex; definition; rules inside regex; test string against it	1
Where to find / generate regexes (regexlib, GenAI)	1.2
Password regex, slashes (no quotes), caret and dollar	2
`test()` function and `!` for invalid	3
Password validation on submit with else if and isValid	4
Password validation on keyup, no isValid needed	5
Phone number (+91, 10 digits, 6-9) and email regex	6
Interacting with a server; async; no reload (Flipkart, Amazon, YouTube)	7
`fetch`, URL, default GET, config object	8
JSONPlaceholder and the users data	9
Promise from fetch, then/catch, response object contents, status, json()	10
async/await version, 404 / error responses	11
Table with thead / tbody; loop; createElement tr/td; appendChild	12-14
Destructuring; column-number questions; React simplification	14.4, 15
Variable as value	16
Variable name as key	17
Square bracket access	18
Variable value as key	19
Tasks: regex form, signup form, red / green borders, evening session	20

> **Remember:** Checked against the whole transcript from the opening regex discussion to the closing tasks. Items I added beyond the lecture (extra notes, my examples, interview answers) are labelled as such.